

CROSSY ROAD

C++ Edition



Made By: BYTE ME

Zuha Azhar
Areeba

CT-25055
CT-25070

Muhammad Ashar
Ali Hasan

CT-25087
CT-25098

Section B

CSIT 2025

Mr Ahmed Zaki / Dr Murk Marvi

Why Crossy Road? (Problem Statement)

Crossy Road was selected for its architectural complexity, requiring polymorphism, encapsulation, and the Prototype design pattern. It serves as a robust platform for implementing dynamic dispatch and abstraction within a professional software framework.

Why SFML as our GUI library?

SFML's native C++ architecture and RAI support provide a seamless OOP integration for windowing and 2D rendering. Its design allows for custom system building while directly mapping the Sprite/Texture pattern onto our Prototype implementation. Using SFML 3 further introduced real-world constraints, providing experience with modern C++ features like `std::optional`.

OOP Principles :

Encapsulation

Encapsulation in this project bundles data like textures and RNG states within classes, exposing only essential methods like `update()` and `getBounds()` to external code. By declaring fields as `private` or `protected`, it prevents the game loop from accidentally corrupting an object's internal state. The `WorldGenerator` best demonstrates this by hiding a complex generation subsystem behind a simple, single-method public contract.

Inheritance

Inheritance serves as the backbone of the entity system, using an abstract `Obstacle` base class to manage shared traits like textures, speed, and bounds. Concrete subclasses like `Vehicle` and `Log` extend this for movement, while `Tree` and `Rock` implement static behavior. By using pure virtual functions, the system ensures every entity has its own `update()` logic while allowing the collision system to handle different object types through a single interface.

Polymorphism

Polymorphism enables the game loop to manage all entities through a single `Obstacle*` pointer, allowing the `update()` and `draw()` methods to automatically trigger the correct logic for a `Vehicle`, `Log`, or `Tree` at runtime. This "dynamic dispatch" eliminates the need for messy `switch-case` blocks, making the codebase highly scalable and easy to maintain when adding new obstacles. Additionally, it allows the collision system to treat all objects uniformly while using `dynamic_cast` to apply specific behaviors, like riding a `Log` versus crashing into a `Vehicle`.

OOP Principles :

Abstraction

Abstraction simplifies the codebase by focusing on what an object does rather than how it does it. It decouples the game loop from complex logic, such as procedural world generation, audio buffer management, and various collision algorithms, which are hidden behind clean, descriptive method calls. By using abstract contracts like the `Obstacle` class and the `GameState` enum, the project remains modular and easy to manage without "spaghetti code."

Composition

Composition uses "has-a" relationships to model ownership, where one object contains and manages the lifecycle of others. For example, the `WorldGenerator` owns a registry of prototype obstacles, and each `Obstacle` manages its own heap-allocated SFML `Sprite` and `Texture`. In the main game loop, composition is used to manage vectors of active `Lanes` and `Obstacles`, ensuring they are created and destroyed together as part of the game world.

Game Logic:

(solution definition)

Game loop

The game loop adheres to the standard Event → Update → Draw cycle, gating all operations behind the current GameState to separate menu, gameplay, and death screen logic. It relies on a delta-time multiplier derived from `clock.restart().asSeconds()`, ensuring consistent game speed across all frame rates.

World Generation

The WorldGenerator manages infinite scrolling by comparing a `highestLaneY` cursor against a camera-based spawn threshold. If the cursor exceeds the threshold, `spawnLane()` is called to create a new row, advancing the cursor upward by one TILE. Lane types are randomly selected via `pickLaneType()`, which enforces a fair experience by guaranteeing a SAFE lane after a maximum of four consecutive dangerous lanes. Off-screen lanes and obstacles are then despawned, with obstacle pointers carefully deleted to prevent memory leaks.

Architecture Overview

Layer	Components	Responsibility
Entity Layer	Obstacle, Vehicle, Log, Tree, Rock, Lane	Represents all game-world objects. Manages their own state, textures, and rendering. Provides a uniform interface via Obstacle.
World Layer	WorldGenerator	Procedural generation of an infinite scrolling world. Owns the prototype registry, manages spawn/despawn thresholds, and enforces lane type rules.
Logic Layer	Collision	All collision detection and response logic. Stateless static utility — takes game world snapshots and returns results.
Application Layer	main.cpp	Window management, event loop, camera, player movement, game state machine, HUD rendering, score tracking, audio triggering.

Classes & their Roles

Obstacle

Obstacle is the abstract base class that manages all common entity states (Sprite, Texture, speed) and defines core virtual methods like `update` and `draw`. While providing concrete logic for bounds and movement, it enforces a standard contract across all game objects and uses a deep-copying constructor to enable the Prototype pattern.

Vehicle

The Vehicle class extends Obstacle by using the `VehicleType` enum to select three distinct variants: BIKE, RICKSHAW, or DUMPER. It efficiently manages assets with `static textureFor()` and `sizeFor()` methods and implements horizontal movement in its `update()` method, ensuring vehicles wrap around screen edges when moving out of bounds.

Log

The Log class is structurally similar to Vehicle but acts as a platform rather than an obstacle. When a player stands on a log, the game loop reads `log->getSpeedX()` and applies that velocity to move the player each frame. `Log::update()` handles horizontal movement and screen wrapping identically to the Vehicle class.

Tree/Rock

The Tree and Rock classes are static subclasses with empty `update()` methods, existing mainly to provide distinct textures and enable type-checking in the collision system. They block player movement by pre-calculating the player's next position and preventing moves that would cause a collision.

Lane

The Lane class represents a single horizontal strip in the game world, rendered as a colored `RectangleShape` based on its type: SAFE (green grass), ROAD (grey asphalt), or RIVER (blue water). The WorldGenerator uses `getLaneType()` to determine which obstacles should be spawned on each lane.

World Gen

The WorldGenerator manages the infinite game world by maintaining a `highestLaneY` cursor, comparing it against a spawn threshold each frame to determine when new rows are needed. It handles both generating new lanes via `spawnLane()` and despawning off-screen lanes and obstacles. The `lanesSinceLastSafe` counter ensures gameplay remains fair by enforcing a maximum of four consecutive non-safe lanes.



Collision System

Collision Type	Detection Method	When Checked	Result
Static Blockin	Simulate future FloatRect; check intersection with Tree/Rock	Before player move commits	Cancel the move — player stays put
Vehicle Hit	Current FloatRect intersection with all Vehicle instances	Each frame, during update	GameState → DEAD, play death sound
Log Riding	Current FloatRect intersection with Log instances	Each frame, during update	player.move(log.getSpeedX() * dt, 0)
Drowning	Player on RIVER lane AND no Log intersection	Each frame, during update	GameState → DEAD, play death sound

The Collision class implements four distinct detection algorithms to handle interactions: static blocking, vehicle hits, log riding, and drowning. Each frame, it checks for vehicle collisions, applies log velocity if the player is riding, or triggers the drowning state if the player is on a RIVER lane without a log. The static blocking check is particularly interesting, testing a simulated future position to reject illegal moves before they are committed, thus preventing tunneling.



Game Logic:

Sprite & Audio

The system handles texture loading during entity construction with a graceful fallback to a 1x1 white image to prevent crashes if a file is missing. Sprites are automatically scaled to logical dimensions, and direction-aware flipping is achieved efficiently by negating the X scale and shifting the origin rather than loading separate assets. The `AudioManager` manages audio using `std::optional` and pre-loaded buffers, with background music set to loop and sound effects (hop, death) dynamically emplaced to meet the audio lifetime requirements of SFML.

HighScore System

The score is incremented only when the player crosses a new, unique row, determined by comparing the current Y position against the lowest Y value reached so far (`highestY`). For high score persistence, minimal inline functions in `scoreboard.h` read from and overwrite a local `highscore.txt` file, providing a dependency-free single-player leaderboard.

Camera Movement

The camera uses SFML's `View` class to create a hybrid system: it auto-scrolls upward at a constant speed but snaps to the player if they fall too far behind. By switching between this world view and a static default view, the game ensures the HUD (score and high score) remains fixed on the screen while the environment moves.

Build.Bat

Compilation

How all the files are compiled?

The project utilizes a `build.bat` script to orchestrate the MinGW `g++` compilation through a four-step conceptual flow. It starts with preprocessing, where headers are resolved and class declarations are pulled into translation units.

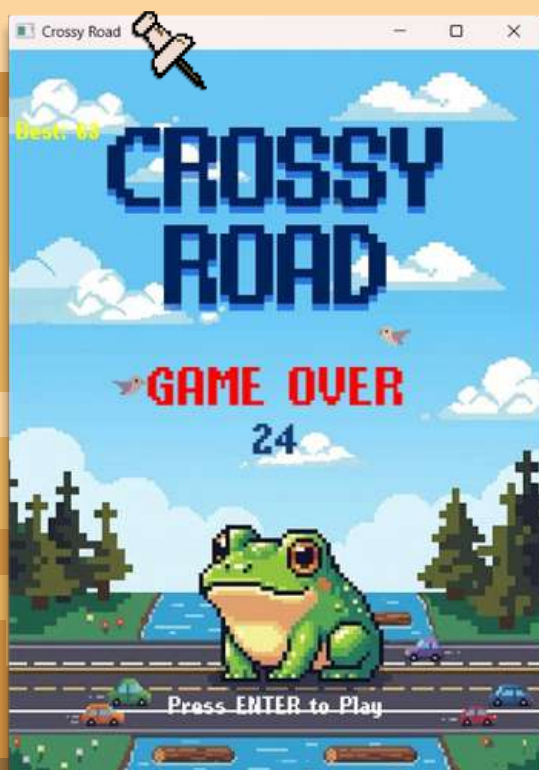
Next, the compilation phase transforms each `.cpp` file into an independent `.o` object file containing machine code. The process concludes with linking, where the linker resolves cross-file references and integrates SFML libraries to generate the final `CrossyRoad.exe` executable.



Game Snippets



Game Snippets



File Structure

CrossyRoad

include/

- audio.h
- collision.h
- constants.h
- lane.h
- movingobstacle.h
- obstacle.h
- scoreboard.h
- staticobstacle.h
- worldgenerator.h

src/

- audio.cpp
- collision.cpp
- constants.cpp
- lane.cpp
- movingobstacle.cpp
- obstacle.cpp
- scoreboard.cpp
- staticobstacle.cpp
- worldgenerator.cpp

assets/

- images/
(stores all the sprites)
- sounds/
(stores all the sounds .wav files)
- fonts/
(stores our custom pixel font)

**highscore.txt/
lib/**

- stores all the sfml library binary files
- all the .lib/
- all the .dll/
- Highscore.txt stores the

build.bat/

- It contains the build script used to compile all the files.
- path to all the source files and the libraries



Identifying Design Pattern

1) Prototype Pattern

The Prototype pattern decouples the WorldGenerator from complex constructor logic by using a virtual clone() method in the Obstacle base class. Pre-configured instances are stored in a registry map, allowing the generator to spawn entities simply by copying a prototype (e.g., "bike") and setting its position. This centralized approach makes the system highly extensible, as adding new obstacle types requires zero changes to the spawning logic.

2) Singleton Pattern

The Collision class is implemented as a static utility, providing a centralized namespace for collision algorithms without requiring instantiation. By making all methods static and eliminating instance state, the design treats collision detection as a stateless service. This approach makes dependencies explicit and ensures the class is easily testable in isolation, as the main game loop can call methods like Collision::checkVehicleHit() directly.



LIMITATIONS



Raw Pointer Management:

Manual new/delete usage creates memory leak risks that `std::unique_ptr` would resolve.

No Difficulty Sealing

Consistent obstacle speed and density prevent the game from becoming harder as the player progresses.

Static Obstacle Overlap

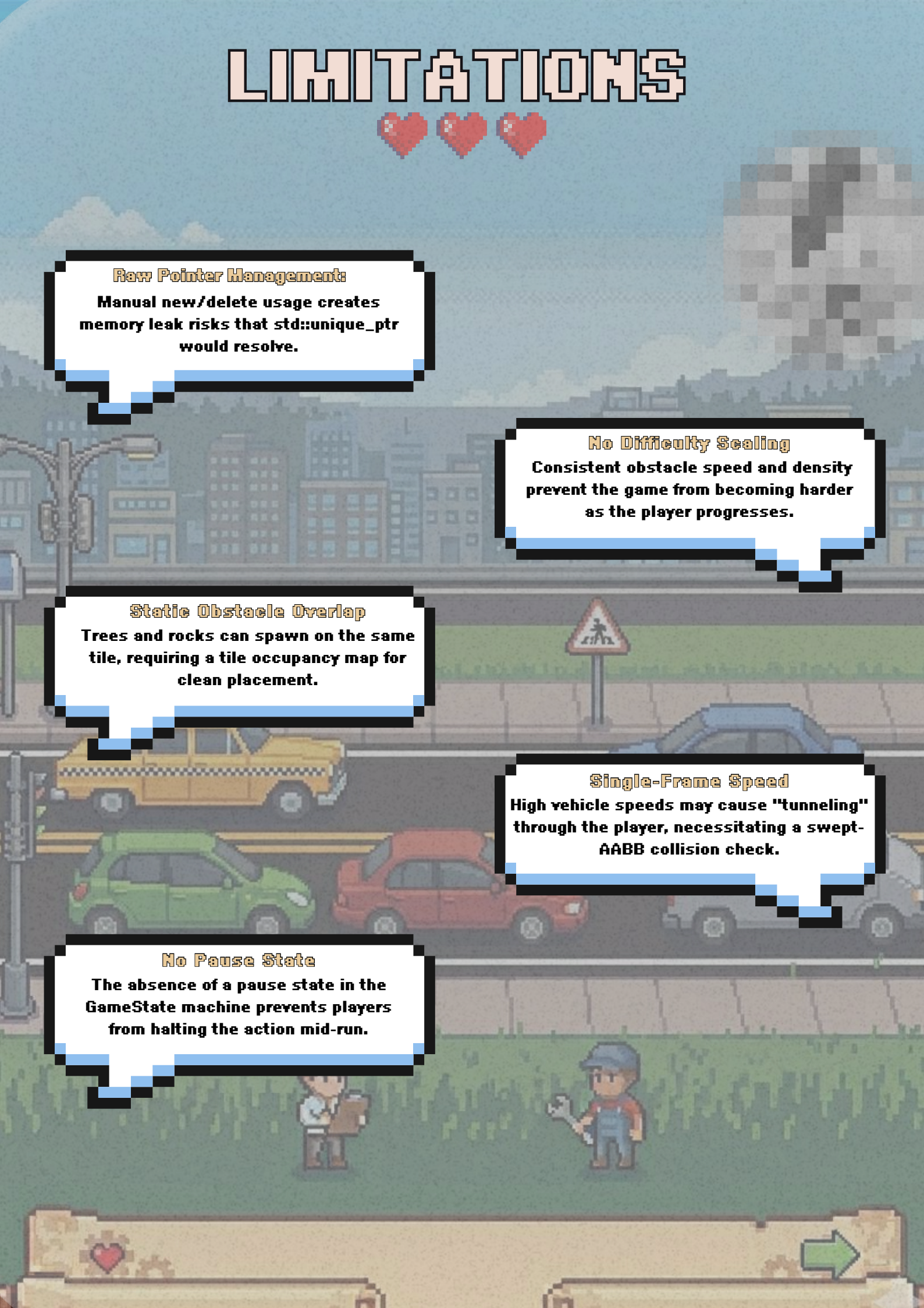
Trees and rocks can spawn on the same tile, requiring a tile occupancy map for clean placement.

Single-Frame Speed

High vehicle speeds may cause "tunneling" through the player, necessitating a swept-AABB collision check.

No Pause State

The absence of a pause state in the GameState machine prevents players from halting the action mid-run.



IMPROVEMENTS

Train Obstacle:

Adding a Train entity using a nested State pattern (alternating between Moving and Stopped) would introduce high-speed, timed hazards for better gameplay variety.

Difficulty Progression:

Implementing a score-based multiplier to scale cameraSpeed and vehicle velocity would transform the static experience into an escalating challenge.

Smart Pointers:

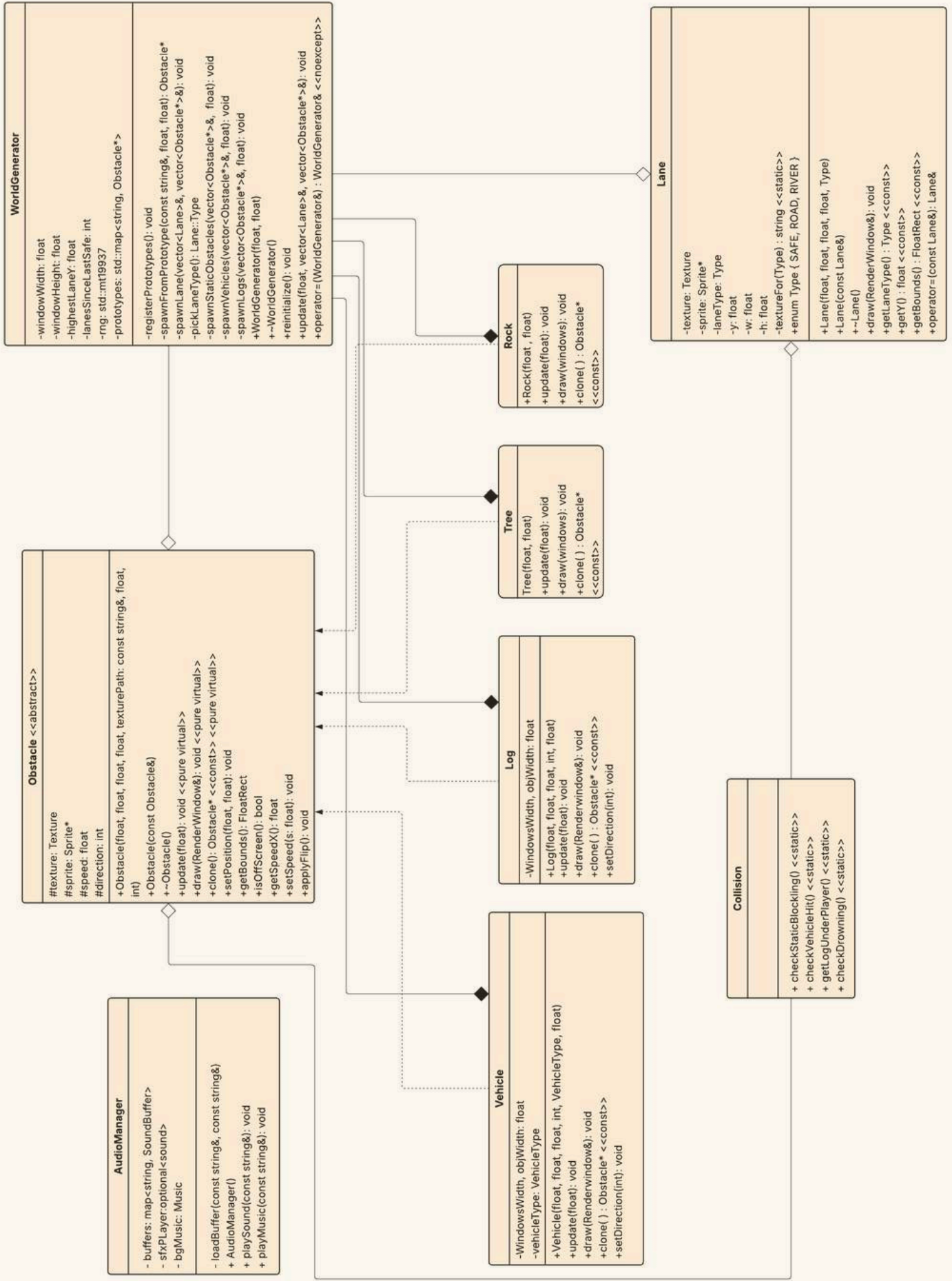
Replacing raw Obstacle* with std::unique_ptr would automate memory management, eliminating manual delete calls and the risk of memory leaks.

Persistent Leaderboard:

Expanding the high score into a structured top-10 list with a dedicated UI state would provide more meaningful competition for players.

Contribution Overview

Name	Contribution	Explanation
Ashar	Teamlead, Engine & Audio Lead	Established the SFML3 environment, core camera, and sprite systems. He handled the player's physical constraints and the entire audio profile (BGM and SFX).
Zuha	Logic & Systems Architect	Developed the WorldGenerator and vehicle mechanics. She implemented the "Prototype" design pattern and handled data persistence for high scores.
Ali	UI & Asset Designer	Focused on the visual interface and static environment. He integrated pixel fonts and HUD elements while bridging the technical and descriptive sides via UML and reporting.
Areeba	Environment & Docs Lead	Managed the compiler configuration and complex "log riding" mechanics. She designed the sprites and served as the primary editor for the project documentation.



UML Relations:

Operator Overloading

In C++, Operator Overloading defines how your classes handle being assigned to one another, ensuring that memory is managed safely. These overloads prevent "shallow copies," where two objects accidentally point to the same memory, leading to crashes when one is deleted.

Specifically, in your diagram, this is happening as follows:

- **Lane (Copy Assignment):** Triggered by `lane1 = lane2`. It performs a deep copy by allocating a new Sprite in memory for the destination object, ensuring every lane owns its own unique graphic.
- **WorldGenerator (Move Assignment):** Triggered by `worldGen = WorldGenerator{...}`. It transfers ownership (steals) resources from a temporary object, which is faster than copying because it simply moves pointers instead of duplicating data.
- **Safety Logic:** Both methods ensure that old memory is cleaned up (deleted) before the new data is assigned, preventing memory leaks during a game reset or update.

Composition

In this system, composition occurs between the Owner classes on the left and the Resource classes on the right. Specifically, the Lane and Obstacle classes own their respective Texture and Sprite objects; the AudioManager owns its collection of SoundBuffer and Music files; and the WorldGenerator owns the Obstacle prototypes. The "filled diamond" indicates that these parent classes are physically responsible for the memory of their components, meaning when an object like a Lane is deleted, its destructor explicitly triggers the deletion of its Sprite and Texture to ensure no data is left orphaned in memory.

- **In the Lane and Obstacle classes:** It happens when they manage their own Texture and Sprite pointers, ensuring these are deleted when the lane or obstacle is removed.
- **In the AudioManager:** It happens as the manager holds a "map" of SoundBuffer and Music files, controlling their existence in the game's memory.
- **In the WorldGenerator:** It happens where the generator owns a "prototypes map" of Obstacles, meaning it is the sole entity responsible for clearing those templates out of RAM.

UML Relations:

Aggregation

In this diagram, Aggregation (the hollow diamond) occurs where a class uses or references an object but does not own its lifecycle. Unlike composition, the "whole" does not delete the "part" when it dies; instead, objects like Lane and Obstacle are created elsewhere and simply passed into these classes as references. This allows multiple systems—like generation and collision—to work with the same game data simultaneously without any single one being responsible for permanently deleting that data from memory.

- **WorldGenerator to Lane/Obstacle:** The generator receives vectors by reference to fill them with game objects, but it doesn't "own" the vectors; they exist independently of the generator's lifecycle.
- **Collision to Lane/Obstacle:** The collision system uses these same references to check for hits. If the Collision object is destroyed, the Lanes and Obstacles stay alive because it only had a "hollow" connection to them.
- **Vector passed by ref:** This is the technical "how"—by using references (&), these classes are just "borrowing" access to the data rather than holding a private, permanent copy.

Association

In this diagram, Association (the solid line with an arrow) represents a general "using" relationship where one object interacts with another to perform its duties without a strict parent-child ownership. It is the most common connection in UML, indicating that a class simply knows about and communicates with another. Unlike composition or aggregation, association doesn't imply a "part-of" hierarchy; it just means the two classes are connected during the normal flow of the program.

- **main to WorldGenerator, AudioManager, and Collision:** The entry point of your program (main) holds and uses these major systems to run the game loop. It calls their functions to update the world or play sounds.
- **main to Lane and Obstacle (vectors):** Your main function maintains the storage (vectors) for the game objects. It associates with them to manage the overall game state.
- **The Arrow Direction:** The arrow points from main toward the other classes, meaning main knows about them and can send them messages, but those classes don't necessarily need to know about main.